



쿠버네티스 기초 실습

아키텍처 이해 · Minikube 설치 · 기본 리소스

2일차 · 오후

실습 환경: Minikube (1일차 실습 VM 그대로 사용)

PART 1

쿠버네티스 아키텍처

클러스터를 구성하는 컴포넌트와
선언적 운영의 동작 원리를 다룬다.



오늘 오후 만들 것

오전에 제기된 다섯 가지 과제의 해결자

배치

수십 대 노드에 컨테이너를 적절히 분산 배치 → kube-scheduler

장애 복구

장애 발생 시 자동 감지 · 재기동 → controller-manager + kubelet

부하 대응

부하에 따른 컨테이너 수 자동 증감 → ReplicaSet (오늘) + HPA (참고)

무중단 배포

서비스 중단 없는 순차 교체 → Deployment 롤링 업데이트

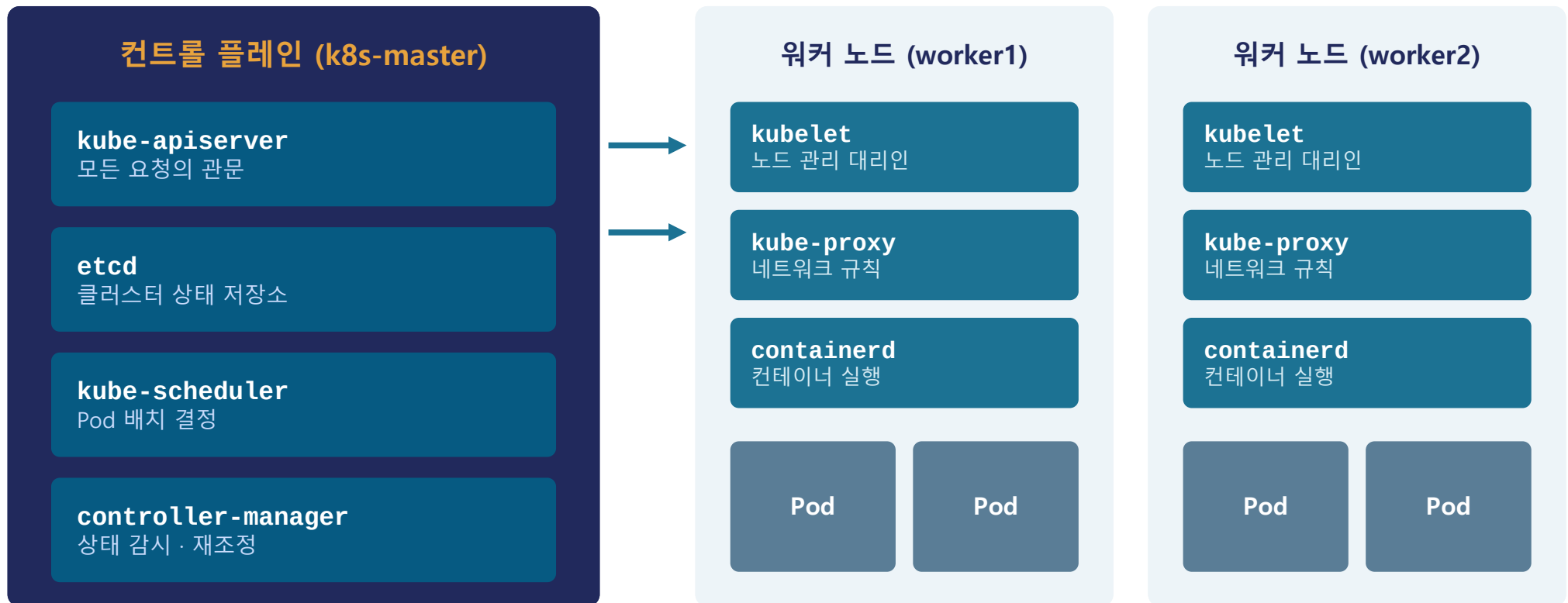
통신 · 주소 관리

동적으로 생성 · 소멸하는 컨테이너의 주소 관리 → Service

오늘의 목표 이 다섯 컴포넌트를 직접 구축하고, 각각이 실제로 문제를 해결하는 모습을 실습으로 확인한다.

클러스터 전체 구조

컨트롤 플레인(두뇌)과 워커 노드(손발)



kubectl → apiserver → (scheduler 결정) → 각 노드 kubelet이 Pod 실행 — 모든 통신은 apiserver를 경유한다.

오늘 실습할 형태: Minikube

여러 상자를 노드 하나에 담는다



핵심 Minikube는 '진짜 K8s'다 — 컴포넌트를 단순화하거나 훔쳐 낸 것이 아니라, 같은 컴포넌트를 노드 하나에 모아 실행할 뿐이다.

컨트롤 플레인 구성요소

클러스터의 두뇌 — 4가지 역할 분담



kube-apiserver

유일한 출입구. kubectl · 컴포넌트 간 모든 통신이 REST API로 여기를 경유하며 인증 · 인가를 수행



etcd

클러스터의 모든 상태를 저장하는 분산 키-값 저장소. 이것이 곧 클러스터의 원본 데이터 — 백업 대상 1순위



kube-scheduler

새 Pod를 어느 노드에 배치할지 결정. 자원 여유 · 제약 조건을 계산해 최적 노드 선택



controller-manager

선언된 상태와 현재 상태를 비교해 차이를 메우는 컨트롤러들의 집합 (복제 수 유지, 노드 감시 등)

핵심 컴포넌트끼리 직접 대화하지 않는다 — 모두 apiserver를 통해 etcd의 상태를 읽고 쓴다.

etcd 심화

클러스터의 유일한 원본 데이터

저장 내용

모든 리소스(Pod · Deployment · Service 등)의 명세와 상태 전체 — 사실상 클러스터 그 자체

홀수 개로 구성하는 이유

합의(Raft) 알고리즘 특성상 과반 투표가 필요 — 3대 또는 5대 구성이 표준 (본 실습은 학습용 1대)

백업의 중요성

etcd 백업이 곧 클러스터 백업 — snapshot save 명령으로 정기 백업하는 것이 운영 표준

백업 명령 참고 `ETCDCTL_API=3 etcdctl snapshot save backup.db` — 운영 환경에서 정기적으로 실행하는 백업 명령의 형태다.

워커 노드 구성요소

Pod가 실제로 실행되는 곳



kubelet

각 노드의 대리인. apiserver에서 할당받은 Pod 명세를 컨테이너 런타임에 지시하고, 상태를 지속 보고



kube-proxy

Service의 가상 IP로 온 트래픽을 실제 Pod로 전달하는 네트워크 규칙 iptables) 관리 — 오전의 DNAT와 동일 원리



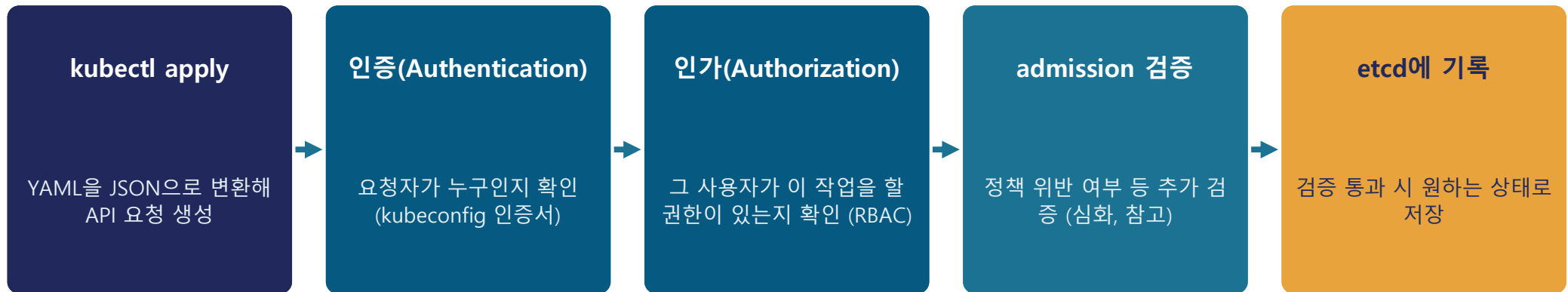
컨테이너 런타임

컨테이너를 실제 생성·실행하는 엔진. 본 실습은 containerd 사용 — 도커 없이 K8s가 CRI로 직접 통신

1일차 연결 "K8s는 도커 CLI 없이 containerd와 직접 통신한다" — 표준(CRI) 덕분에 런타임 교체가 자유롭다.

kubectl 요청의 처리 과정

명령 하나가 클러스터 안에서 거치는 단계



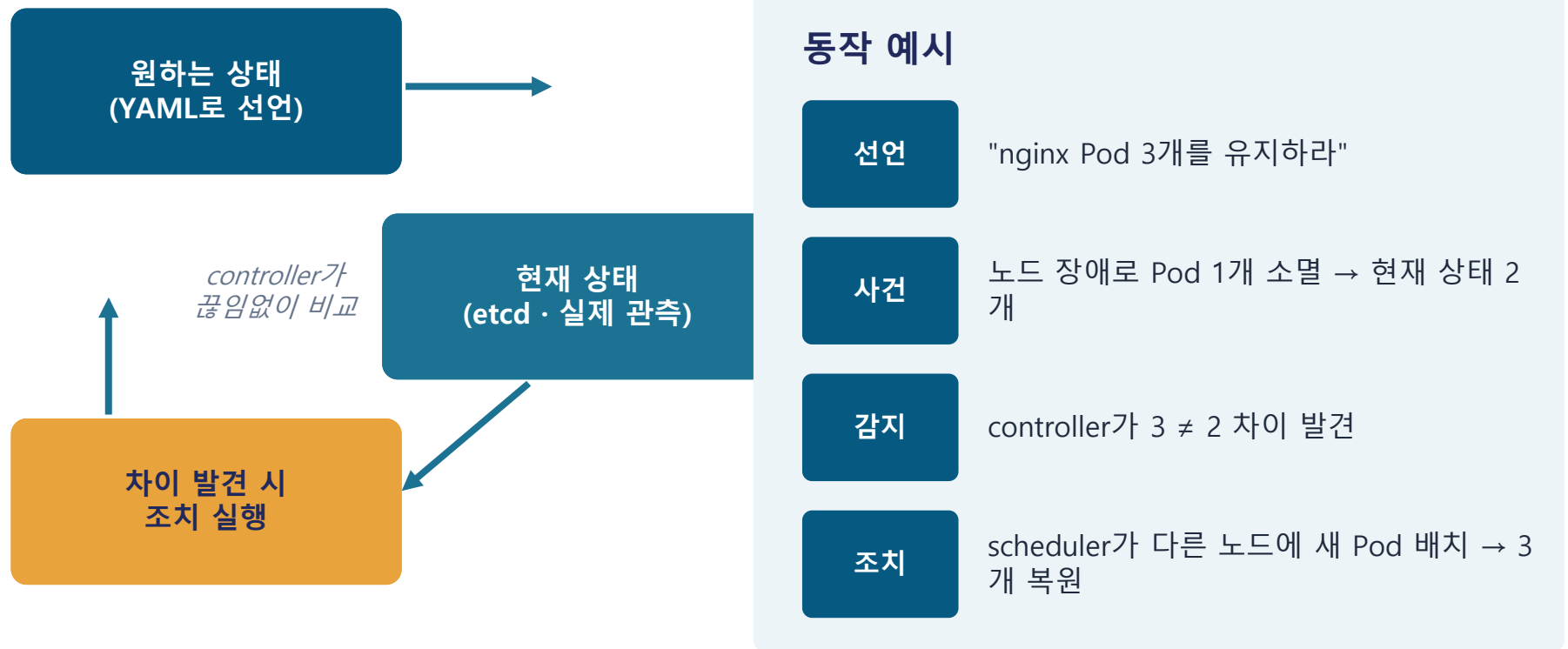
kubeconfig ~/.kube/config에 클러스터 주소 · 인증서가 담겨 있다 — 이것이 곧 kubectl의 '신분증'이다.

주의 kubeconfig 파일이 유출되면 클러스터 전체 접근 권한이 유출되는 것과 같다 — 3일차 CI/CD 시크릿 관리와 연결된다.

RBAC(역할 기반 접근 제어)는 심화 주제로 본 과정 범위 밖이나, 존재와 목적만 인지하고 넘어간다.

선언적 운영과 재조정 루프

쿠버네티스가 스스로 복구하는 원리



명령적 "이렇게 하라"가 아니라 선언적 "이 상태를 유지하라" — 장애 복구가 운영자의 일이 아닌 시스템의 일이 된다.

다양한 컨트롤러들

재조정 루프는 하나의 패턴, 컨트롤러는 여러 종류

컨트롤러	감시 대상	본 교육에서
ReplicaSet	"Pod N개 유지"	오늘 Deployment를 통해 사용
Deployment	ReplicaSet + 배포 이력(롤링업데이트 · 롤백)	오늘 직접 실습
DaemonSet	"모든 노드에 1개씩" (예: 로그 수집 에이전트)	참고 — kube-flannel이 이 방식으로 동작 중
Job / CronJob	"한 번 실행하고 끝" · "정기적으로 실행"	참고 — 배치 작업에 사용

핵심 구현 방식은 다양해도 원리는 동일 — "원하는 상태"를 정의하고 컨트롤러가 이를 지속적으로 실현한다.

PART 2

Minikube로 빠르게 시작하기

제한된 실습 시간 안에 쿠버네티스를 직접 다뤄보기 위해
Minikube로 로컬 클러스터를 구성한다.



왜 Minikube인가

학습 목적에 맞는 가장 빠른 시작점



설치 시간

명령 한 줄(minikube start)로 수 분 내 완료 — kubeadm 멀티노드 구축(1시간 이상) 대비 압도적으로 빠르다



동일한 kubectl 경험

Pod · Deployment · Service를 다루는 명령과 개념은 운영 클러스터와 완전히 동일하다



별도 VM 불필요

1일차부터 쓰던 VM에 이미 설치된 Docker를 그대로 드라이버로 사용 — 새 VM을 만들 필요가 없다

실무에서는 실제 다중 노드 클러스터 구축(kubeadm) 절차도 알아둘 가치가 있다 — 관련 자료는 별도로 배포하니 시간 될 때 참고한다.

Minikube란 무엇인가

로컬 환경에 K8s 클러스터를 만드는 도구

한 대의 컴퓨터(VM) 안에 노드 1개짜리 쿠버네티스 클러스터를 만들어주는 공식 도구. 컨테이너(Docker) 또는 가상머신 드라이버 위에서 실제 kubeadm과 동일한 방식으로 클러스터를 초기화한다.

드라이버	동작 방식	오늘 실습
docker	노드를 하나의 도커 컨테이너로 실행	채택 — 이미 설치된 Docker 재사용
kvm2 / virtualbox	노드를 별도의 경량 VM으로 실행	참고 — 중첩 가상화 설정 필요
none	호스트에 직접 설치 (격리 없음)	참고 — 실습 환경에는 부적합

오늘의 선택 docker 드라이버 — VM 안에 또 VM을 띄우는 중첩 가상화 없이, 이미 설치된 Docker 위에 바로 클러스터가 올라간다.

사전 준비 확인

1일차에 이미 끝나 있는 것들

확인 명령

```
# Docker가 정상 동작하는지 확인 (1일차 설치)
docker version
docker ps

# 현재 사용자가 docker 그룹에 속해 있는지 확인
groups | grep docker
```



이미 준비됨

1일차 오후에 설치한 Docker를 그대로 사용한다.

별도 VM이나 추가 하이퍼바이저 설정 없이 바로 다음 단계(kubectrl · Minikube 설치)로 진행할 수 있다.

주의 docker 그룹에 속해 있지 않다면 1일차의 `usermod -aG docker $USER` 및 재로그인을 먼저 완료한다.

kubectl 설치

클러스터를 다룰 명령줄 도구부터

kubectl 설치

```
# 최신 안정 버전 다운로드
curl -LO "https://dl.k8s.io/release/$(curl -L -s \
https://dl.k8s.io/release/stable.txt)/bin/linux/\
amd64/kubectl"

# 실행 권한 부여 및 설치
chmod +x kubectl
sudo mv kubectl /usr/local/bin/

# 확인
kubectl version --client
```



kubectl의 위치

kubectl은 클러스터의 종류(Minikube 든 실제 운영 클러스터든)와 무관하게 동일하게 동작하는 관리 도구다.

오늘 배우는 kubectl 사용법은 어떤 K8s 환경에서도 그대로 유효하다.

Minikube 설치

공식 바이너리 다운로드

Minikube 설치

```
# 최신 바이너리 다운로드
curl -LO https://storage.googleapis.com/\
  minikube/releases/latest/\
  minikube-linux-amd64

# 실행 권한 부여 및 설치
sudo install minikube-linux-amd64 \
  /usr/local/bin/minikube

# 확인
minikube version
```



설치 방식

Minikube는 단일 실행 파일(바이너리)이다.

별도 패키지 저장소 등록 없이 다운로드 후 실행 권한만 부여하면 바로 사용할 수 있다 — 도커 설치보다 훨씬 간단하다.

실습 ① Minikube 클러스터 시작

명령 한 줄로 클러스터 기동

클러스터 시작

```
# Docker 드라이버로 클러스터 시작 (수 분 소요)  
minikube start --driver=docker
```

```
* minikube v1.33.x on Ubuntu 22.04  
* Using the docker driver  
* Creating docker container ...  
* Preparing Kubernetes v1.30.x ...  
* Verifying Kubernetes components...  
* Done! kubectl is now configured
```

내부적으로 하는 일

도커 컨테이너 1개를 노드처럼 만들고, 그 안에서 kubeadm init과 사실상 동일한 초기화를 수행한다

kubeconfig 자동 설정

완료 메시지의 "kubectl is now configured"처럼 별도 설정 없이 바로 kubectl 사용 가능

실습 ② 클러스터 상태 확인

Part 1 이론과 실물 대조

상태 확인

```
# 노드 확인 (1개만 있음 - Ready 상태)
kubectl get nodes
NAME          STATUS    ROLES    VERSION
minikube      Ready     control-plane  v1.30.x

# 시스템 컴포넌트 Pod 확인
kubectl get pods -n kube-system
etcd-minikube      Running
kube-apiserver-minikube  Running
kube-scheduler-minikube  Running
coredns-...        Running

# Minikube 자체 상태
minikube status
```



Part 1과의 대조

kube-system 네임스페이스의 Pod 목록이 오늘 오전(2일차 오전 자료) 배운 컨트롤 플레인 컴포넌트와 정확히 일치한다.

노드는 1개뿐이지만 역할(control-plane)에 워커 기능까지 포함되어 있다.

보너스: Minikube 대시보드

브라우저로 클러스터 들여다보기

대시보드 실행

```
# 웹 기반 대시보드 실행
minikube dashboard --url
http://127.0.0.1:port/... (URL 출력)
```



활용

Pod · Deployment · Service 등 리소스를 그래픽 화면으로 조회할 수 있다. kubectl 명령 결과를 시각적으로 재확인할 때 유용하다.

실습 팁 이후 실습에서 kubectl get/describe로 확인한 내용을 대시보드에서 한 번씩 대조해 보면 리소스 관계를 이해하는 데 도움이 된다.

Minikube 트러블슈팅과 정리

자주 쓰는 명령 모음

상황	명령	설명
시작 실패	<code>minikube start --alsologtostderr -v=1</code>	상세 로그와 함께 재시도, 오류 원인 확인
완전히 꼬였을 때	<code>minikube delete && minikube start --driver=docker</code>	삭제 후 새로 시작 (수 분이면 복구)
일시 중지	<code>minikube stop</code>	리소스 절약, 다음에 <code>minikube start</code> 로 재개
상태 확인	<code>minikube status / kubectl cluster-info</code>	클러스터 · 각 컴포넌트 상태 점검

스냅샷보다 간편한 복구 kubeadm 멀티노드와 달리 `minikube delete` → `start`는 수 분이면 끝난다 — VM 스냅샷 없이도 이 명령만으로 충분히 빠르게 복구된다.

PART 3

기본 리소스 실습

Pod · Deployment · Service —
쿠버네티스 운영의 최소 단위 세 가지를 다룬다.



Pod — 배포의 최소 단위

컨테이너를 감싸는 쿠버네티스의 기본 상자



Pod 단위의 의미

K8s는 컨테이너를 직접 다루지 않고 Pod 단위로 생성 · 배치 · 삭제한다. 대부분의 Pod는 컨테이너 1개

Pod마다 고유 IP

포트 매핑 없이 Pod IP로 직접 통신 — 도커 -p와의 차이 (상세는 3일차 네트워크)

일회성 존재

Pod는 죽으면 새로 만들어지며 IP도 바뀐다 — 그래서 Deployment와 Service가 필요

셋째 특성(일회성)이 이후 모든 리소스 설계의 출발점이다.

Pod의 생명주기 상태

Phase로 확인하는 Pod의 현재 단계

Phase	의미	흔한 원인(비정상 시)
Pending	스케줄링 대기 또는 이미지 다운로드 중	자원 부족, 이미지 pull 지연
Running	최소 1개 컨테이너가 정상 실행 중	정상 상태
Succeeded	모든 컨테이너가 정상 종료 (재시작 없음)	Job 등 일회성 작업에서 정상
Failed	컨테이너가 오류로 종료됨	앱 오류, 잘못된 명령
Unknown	노드와 통신 불가로 상태 파악 불가	노드 장애 · 네트워크 단절

확인 명령 `kubectl get pods`의 STATUS 열이 이 Phase를 보여준다 — CrashLoopBackOff 등 세부 상태는 컨테이너 상태(재시작 이력)를 반영한 표시다.

실습 ① 첫 Pod 배포

YAML 작성 → apply → 관찰

Pod 정의 YAML

```
# nginx-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.25
    ports:
    - containerPort: 80
```

배포 명령

```
# 배포 및 관찰
kubectl apply -f nginx-pod.yaml
kubectl get pods -o wide
NAME          READY   STATUS    IP
nginx-pod     1/1     Running   10.244.1.5
              NODE: minikube

# 클러스터 내부에서 Pod IP 접근 확인
kubectl run test --rm -it \
  --image=busybox -- wget -qO- 10.244.1.5
```

YAML 4대 필드 apiVersion(스키마 버전) · kind(리소스 종류) · metadata(이름 · 라벨) · spec(원하는 상태) — 모든 리소스가 이 구조를 따른다.

labels의 app: nginx는 이후 Service가 Pod를 찾는 연결 고리가 된다 — 기억해 둘 것.

실습 ② describe로 Pod 상세 확인

장애 분석의 출발점, Events 읽기

describe 출력

```
$ kubectl describe pod nginx-pod
Name:          nginx-pod
Node:          minikube/192.168.49.2
Status:        Running
IP:            10.244.1.5
Containers:    nginx  Image: nginx:1.25  State: Running

Events:
  Type     Reason      Age    Message
  ----     -
  Normal   Scheduled   2m     Successfully assigned to minikube
  Normal   Pulled      2m     Container image already present
  Normal   Started     2m     Started container nginx
```

Events 읽는 법 시간순으로 Scheduled → Pulled → Created → Started가 정상 순서 — 중간에 Warning이 있으면 그 지점이 문제의 원인이다.

kubectl 핵심 명령

도커 명령과의 대응으로 빠르게 익히기

작업	docker (1일차)	kubectl
목록 조회	docker ps	kubectl get pods (-o wide, -A)
상세 정보	docker inspect	kubectl describe pod <이름>
로그 확인	docker logs -f	kubectl logs -f <파드>
내부 접속	docker exec -it ... bash	kubectl exec -it <파드> -- bash
삭제	docker rm -f	kubectl delete pod <이름> / -f 파일.yaml
리소스 생성	docker run ...	kubectl apply -f 파일.yaml (선언적)

1일차 대응 왼쪽 열의 도커 명령을 이미 알고 있다면, 오른쪽 kubectl 명령은 사실상 이름만 바뀐 것에 가깝다.

YAML 작성을 빠르게 하는 팁

매번 손으로 치지 않는 방법

YAML 생성 팁

```
# 1) 필드 문서를 그 자리에서 확인
kubectl explain pod.spec.containers

# 2) 실행하지 않고 YAML만 생성 (--dry-run)
kubectl create deployment sample \
  --image=nginx --dry-run=client -o yaml \
  > sample.yaml

# 3) 생성된 템플릿을 열어 필요한 부분만 수정
vim sample.yaml
```



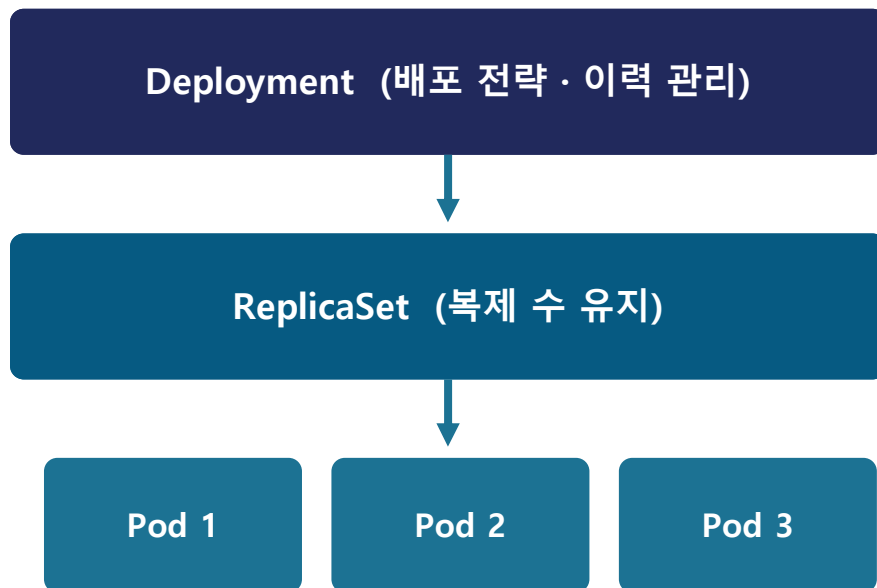
실무 습관

완전히 처음부터 YAML을 작성하는 경우는 드물다.

--dry-run=client -o yaml로 뼈대를 만들고 필요한 부분(replicas, 라벨 등)만 고치는 방식이 훨씬 빠르고 오타도 적다.

Deployment — Pod의 관리자

복제 수 유지와 버전 배포를 담당



replicas 유지

"3개 유지" 선언 → Pod가 죽으면 ReplicaSet이 즉시 보충 (재조정 루프의 실행자)

롤링 업데이트

이미지 변경 시 새 ReplicaSet을 만들어 점진 교체 — 무중단 배포

롤백

이전 ReplicaSet이 이력으로 남아 rollout undo 한 명령으로 복귀

실무 원칙 Pod를 직접 만들지 않는다 — 항상 Deployment로 관리한다.

실습 ③ Deployment 배포

3개 복제로 웹 서비스 배포

Deployment 정의 및 배포

```
# web-deploy.yaml (요지)
apiVersion: apps/v1
kind: Deployment
metadata: { name: web }
spec:
  replicas: 3
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
        - { name: web, image: nginx:1.25 }

kubectl apply -f web-deploy.yaml
```

결과 확인

```
kubectl get pods -o wide
web-a1   Running   minikube
web-b2   Running   minikube
web-c3   Running   minikube

# 배포 상태 요약
kubectl get deploy web
READY 3/3   UP-TO-DATE 3
```

실습 ④ 셀프 힐링 관찰

Pod를 강제로 지워도 자동 복구되는가

강제 삭제 실험

```
# Pod 하나를 강제로 삭제
kubectl get pods -l app=web
kubectl delete pod <web-a1과 같은 이름>

# 실시간 관찰 (-w = watch)
kubectl get pods -w
web-a1 Terminating
web-d4 Pending
web-d4 ContainerCreating
web-d4 Running # 수 초 내 보충 완료
```



관찰의 의미

운영자가 복구한 것이 아니라 컨트롤러가 선언(3개)과 현실(2개)의 차이를 메운 것.

Part 1에서 배운 재조정 루프가 지금 눈앞에서 실행되고 있다 — 이것이 K8s의 핵심 가치다.

실습 ⑤ 롤링 업데이트

무중단 버전 교체

업데이트 명령

```
# 1) 이미지 버전 변경 (1.25 → 1.27)
kubectl set image deployment/web web=nginx:1.27

# 2) 교체 과정 실시간 관찰
kubectl rollout status deployment/web
Waiting... 1 out of 3 new replicas updated
deployment "web" successfully rolled out

# 3) 배포 이력 확인
kubectl rollout history deployment/web
```



동작 방식

새 버전 Pod를 1개 만들고 준비되면
구버전 1개 제거 — 이 과정을 반복하
여 서비스 중단 없이 교체한다.

1일차 실습(v1 · v2 이미지 공존)이 바
로 이 기능의 토대다.

실무에서는 set image 대신 YAML 수
정 + apply가 원칙.

실습 ⑤ 롤링 업데이트

무중단 버전 교체

업데이트 명령

```
# 1) 이미지 버전 변경 (1.27 → 변경)
kubectl annotate deployment/web kubernetes.io/change-
cause="버전 수정"
kubectl set image deployment/web web=nginx

# 2) 교체 과정 실시간 관찰
kubectl rollout status deployment/web
Waiting... 1 out of 3 new replicas updated
deployment "web" successfully rolled out

# 3) 배포 이력 확인
kubectl rollout history deployment/web
```

주석넣기

annotate을 사용해 어떤 부분을
업데이트 했는지 주석을 넣어 이력을
관리할 수 있다.

실습 ⑥ 롤백과 배포 전략 옵션

이전 버전으로 즉시 복귀

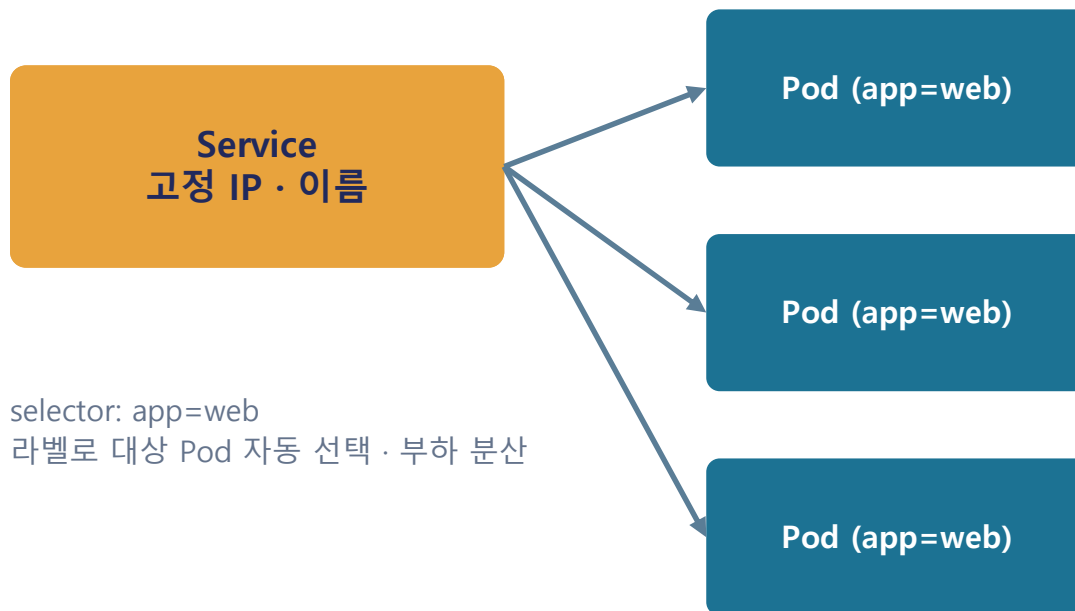
롤백 명령

```
# 직전 버전으로 롤백
kubectl rollout undo deployment/web
kubectl describe deploy web | grep Image
Image: nginx:1.25 # 복귀 완료

# 특정 리비전으로 롤백 (이력 중 선택)
kubectl rollout undo deployment/web \
  --to-revision=1
```

Service — 변하지 않는 접점

일회성 Pod들 앞의 고정 창구



타입	용도
ClusterIP (기본)	클러스터 내부 통신 전용
NodePort	노드IP:포트로 외부 노출 (30000대)
LoadBalancer	클라우드 LB 연동 (온프레미스는 별도 구성)

해결하는 문제 Pod IP는 재생성마다 바뀐다 → Service가 고정 IP · DNS 이름을 제공하고 뒤의 Pod들로 트래픽을 분산한다.

연결 원리는 라벨 셀렉터 — Pod에 붙인 label과 Service의 selector가 일치하면 자동 연결된다.

실습 ⑦ ClusterIP로 내부 통신

클러스터 안에서만 필요한 통신부터

ClusterIP 실습

```
# web-svc-internal.yaml
apiVersion: v1
kind: Service
metadata: { name: web-internal }
spec:
  selector: { app: web }
  ports:
    - { port: 80, targetPort: 80 }

kubectl apply -f web-svc-internal.yaml
kubectl run test --rm -it --image=busybox \
  -- wget -qO- web-internal
```



ClusterIP가 기본값인 이유

대부분의 서비스(DB, 내부 API)는 클러스터 밖에 노출될 필요가 없다.

기본 타입이 ClusterIP인 것은 '필요한 만큼만 노출한다'는 보안 원칙을 반영한다.

실습 ⑧ NodePort로 외부 노출

클러스터 밖에서 접속

Service 정의

```
# web-svc.yaml
apiVersion: v1
kind: Service
metadata: { name: web-svc }
spec:
  type: NodePort
  selector: { app: web }
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

배포 및 접속

```
kubectl apply -f web-svc.yaml
kubectl get svc web-svc
TYPE      PORT(S)
NodePort  80:30080/TCP

# Minikube 전용 접속 방법
minikube service web-svc --url
http://192.168.49.2:30080
curl $(minikube service web-svc --url)
```

포트 3종 구분 port(Service 자체 포트) · targetPort(컨테이너 포트) · nodePort(노드에서 개방되는 포트) — 셋을 혼동하지 않는다.

Service의 실체 — Endpoints

Service 뒤에서 실제로 연결되는 대상 확인

Endpoints 확인

```
$ kubectl get endpoints web-svc
NAME          ENDPOINTS
web-svc       10.244.1.5:80,10.244.1.6:80,10.244.2.4:80

# Pod를 하나 지우면 Endpoints도 자동 갱신됨
kubectl delete pod <web 파드 하나>
kubectl get endpoints web-svc    # 목록이 즉시 갱신
```

핵심 Service는 고정된 것처럼 보이지만, 그 뒤의 Endpoints 목록은 selector와 일치하는 Pod IP들로 실시간 갱신된다.

주의 curl 접속이 안 될 때 Endpoints가 비어 있다면 selector와 Pod의 label이 일치하지 않는 것이 원인이다 — 3일차 트러블슈팅에서 재확인.

네임스페이스 — 논리적 칸막이

환경 · 팀 단위 분리

네임스페이스 명령

```
# 네임스페이스 조회 · 생성
kubectl get namespaces
kubectl create namespace dev

# 특정 네임스페이스에 배포 · 조회
kubectl apply -f web-deploy.yaml -n dev
kubectl get pods -n dev

# 전체 네임스페이스 횡단 조회
kubectl get pods -A
```

용도 환경 분리(dev · staging · prod), 팀 · 업무 단위 분리, 자원 쿼터의 적용 단위

이름 충돌 방지 같은 이름의 리소스도 네임스페이스가 다르면 공존 가능

3일차 연결 Helm의 환경별 분리 배포에서 네임스페이스를 적극 활용한다

참고: 네임스페이스 자원 제한

멀티테넌시 환경의 자원 통제

ResourceQuota 예시

```
# 네임스페이스 전체의 자원 사용 상한 (ResourceQuota 예시)
apiVersion: v1
kind: ResourceQuota
spec:
  hard: { requests.cpu: "4", requests.memory: 8Gi }
```



공공기관 맥락

여러 부서 · 프로젝트가 하나의 클러스터를 공유하는 경우, 한 팀의 과다 사용이 다른 팀에 영향을 주지 않도록 네임스페이스 별로 자원 상한을 설정한다. 본 과정은 단일 사용자 실습이라 직접 다루지 않으나, 운영 확산 시 필수적으로 검토해야 하는 요소다.

라벨과 셀렉터 활용

리소스를 자유롭게 묶고 찾는 방법

라벨 실습

```
# 여러 기준의 라벨 부여
kubectl label pod nginx-pod env=dev tier=frontend

# 라벨로 필터링 조회
kubectl get pods -l env=dev
kubectl get pods -l 'tier=frontend,env=dev'

# 라벨 제거
kubectl label pod nginx-pod env-
```



실무 활용

버전(v1/v2), 환경(dev/prod), 팀 등 다양한 기준으로 라벨을 부여해두면 하나의 명령으로 원하는 대상만 골라 조회 · 배포 · 삭제할 수 있다.

실습 리소스 정리

세션 종료 전 정리

정리 명령

```
# 실습 리소스 일괄 삭제 (kube-system은 건드리지 않는다)
kubectl delete -f web-svc.yaml -f web-svc-internal.yaml \
  -f web-deploy.yaml -f nginx-pod.yaml
kubectl delete ns dev

# 정리 확인
kubectl get all
```

주의 kube-system 네임스페이스의 리소스는 절대 삭제하지 않는다 — 클러스터 자체가 손상된다.

확인 kubectl get all은 default 네임스페이스의 리소스를 한눈에 보여준다 — 정리 후 깨끗한지 여기서 확인한다.

핵심 정리

2일차 오후 주요 내용 요약

- 1 클러스터는 컨트롤 플레인(apiserver · etcd · scheduler · controller)과 워커(kubelet · kube-proxy · 런타임)로 구성된다.
- 2 minikube start 한 줄로 노드 1개짜리 클러스터가 완성된다 — 내부적으로는 kubeadm과 동일한 초기화가 일어난다.
- 3 Pod는 배포의 최소 단위이자 일회성 존재 — 직접 만들지 않고 Deployment로 관리한다.
- 4 Deployment는 복제 유지 · 롤링 업데이트 · 롤백을, Service는 라벨 셀렉터 기반 고정 접점과 부하 분산을 담당한다.
- 5 모든 것은 선언(YAML) → apply → 재조정 루프 — 복구는 운영자가 아닌 시스템의 일이다.

3일차 오전

다음 세션: 쿠버네티스 심화

K8s 네트워크 · Helm 차트 · CI/CD 파이프라인



Ingress와 ConfigMap · Secret — 트래픽과 설정의 관리



Helm 차트 작성과 환경별(dev · prod) 분리 배포



GitHub Actions로 CI/CD 파이프라인 구성

Minikube는 종료하지 않고 유지한다(minikube stop 하지 않기). GitHub 계정 · PAT를 준비해 온다.